

Durée : 16h (4h/semaine en 4 semaines) **Atelier de professionnalisation - Semestre 2**

1. Compétences abordées

- B1.2 Recenser et identifier les ressources numériques
 - B1.3 Développer la présence en ligne de l'organisation
 - B2.1 Concevoir et développer une solution applicative
 - B2.2 Assurer la maintenance corrective ou évolutive d'une solution applicative
-

2. Contexte

L'agence de location "AutoRent"

AutoRent est une agence de location de véhicules qui souhaite moderniser la gestion de son parc automobile. Actuellement, les données des véhicules sont reçues quotidiennement du siège sous forme de fichiers CSV bruts, difficiles à exploiter directement.

L'agence a besoin de deux outils complémentaires :

1. Une **application backend (C#)** pour traiter, valider et structurer les données reçues.
 2. Une **interface web (HTML/CSS/PHP)** pour présenter le catalogue des véhicules disponibles aux clients.
-

3. Cahier des Charges

Le projet doit respecter les contraintes techniques et fonctionnelles suivantes :

A. Gestion des Données (Backend)

- L'application doit être capable de lire un fichier de données brutes (`flotte.csv`) fourni par le siège.
- Les données doivent être stockées en mémoire sous forme d'objets structurés (Programmation Orientée Objet).
- Chaque véhicule est caractérisé par : son immatriculation, sa marque, son modèle, sa catégorie, son prix journalier, son kilométrage et son état.
- L'application doit garantir l'intégrité des données (ex: pas de prix négatif, format d'immatriculation valide).

B. Traitements Automatisés

- L'application doit permettre de filtrer les véhicules selon leur état (Disponible, En maintenance, Loué).
- Elle doit calculer des indicateurs financiers (ex: revenu potentiel total du parc).
- Elle doit générer un fichier d'export propre (`disponibles.csv`) contenant uniquement les véhicules prêts à la location, formaté pour être lu par le site web.

C. Interface Client (Frontend)

- Le site web doit afficher dynamiquement la liste des véhicules disponibles à partir du fichier généré par l'application C#.
 - L'interface doit être moderne et responsive (adaptée aux mobiles).
 - Le client doit pouvoir visualiser clairement les informations de chaque véhicule (Marque, Modèle, Prix, etc.).
-

4. Fonctionnement attendu de l'application C#

L'application backend sera une **application Console** interactive destinée au gestionnaire de parc. Voici le scénario d'utilisation type :

1. **Démarrage** : L'application se lance et charge automatiquement les données depuis le fichier source (`flotte.csv`). Si le fichier est introuvable ou corrompu, un message d'erreur explicite doit s'afficher.
 2. **Menu Principal** : L'utilisateur accède à un menu textuel proposant plusieurs actions :
 - *Afficher le parc complet* : Liste tous les véhicules avec leurs détails.
 - *Filtrer par état* : Demande à l'utilisateur quel état il souhaite voir (ex: "Disponible") et affiche les véhicules correspondants.
 - *Afficher les statistiques* : Affiche le nombre total de véhicules et le revenu potentiel global.
 - *Exporter pour le Web* : Génère le fichier `disponibles.csv` et affiche un message de confirmation.
 - *Quitter* : Ferme l'application.
 3. **Interaction** : L'application doit gérer les erreurs de saisie (ex: choix invalide dans le menu) sans planter.
-

5. Développement de l'Application C# (Partie 2)

Vous devez concevoir et développer l'application en respectant les principes de la **Programmation Orientée Objet**.

Travail à faire :

1. **Analyse et Modélisation** :
 - Identifiez les entités nécessaires au projet. Quelles classes créer pour représenter un véhicule ? Pour gérer l'ensemble du parc ?
 - Définir les propriétés de chaque classe en respectant le principe d'encapsulation (attributs privés, accès contrôlé).
 - Réaliser le **Diagramme de Classes** avant de commencer le développement.
2. **Implémentation** :
 - Développer les classes identifiées.
 - Mettre en place la lecture du fichier CSV. Comment transformer une ligne de texte en un objet C# ?
 - Implémenter la logique du menu principal et des différentes fonctionnalités décrites dans le "Fonctionnement attendu".
 - Gérer l'écriture du fichier d'export. S'assurer que le format de sortie est compatible avec ce que le site web attend.

6. Développement de l'Interface Web (Partie 3)

L'interface web est la vitrine pour les clients. Elle ne doit pas contenir de logique métier complexe (tout est fait en C#), mais se concentrer sur l'affichage.

Travail à faire :

1. Structure de la page (`index.php`) :

- Concevoir une page qui lit le fichier `disponibles.csv`.
- Pour chaque ligne du fichier, générer dynamiquement le code HTML nécessaire pour afficher une "carte" véhicule.

2. Design et Ergonomie (CSS) :

- Mettre en page les résultats de manière grille (Grid ou Flexbox).
- S'assurer que le site est lisible sur mobile.
- Utiliser des codes couleurs pour rendre l'information claire (ex: prix en évidence).

3. Recherche (Bonus) :

- Ajouter un champ de recherche permettant de filtrer l'affichage (ex: par marque) sans recharger la page (JavaScript) ou avec rechargement (PHP).

7. Livrables attendus

- **Le Code Source C#** (Projet Visual Studio complet, code commenté).
- **Le Code Source Web** (Fichiers PHP, CSS).
- **Le Diagramme de Classes** (Format image ou PDF).
- **Un Guide Utilisateur** (PDF) expliquant comment utiliser l'application C# pour mettre à jour le site web.

8. Ressources et Aides

• Format du fichier CSV source (`flotte.csv`) :

```
Immatriculation;Marque;Modele;Categorie;Prix;Km;Etat
AB-123-CD;Peugeot;208;Citadine;35;12000;Disponible
EF-456-GH;Renault;Clio;Citadine;32;45000;En maintenance
IJ-789-KL;BMW;X5;SUV;120;50000;Loué
```

• Conseils Techniques :

- Pour la lecture/écriture de fichiers en C#, se renseigner sur les classes du namespace `System.IO`.
- Pour le CSV en PHP, il existe des fonctions natives très pratiques pour lire un fichier ligne par ligne.
- Penser à gérer les exceptions (fichiers manquants, erreurs de conversion de types).

Note : Ce projet simule une architecture réelle où un "Back-Office" (C#) prépare les données pour un "Front-Office" (Web). Soigner la communication entre les deux (le format du fichier CSV d'échange).